

Week 4 Day 3

1. Self Join

It is a regular join, but the table is joined with itself.

2. Union Operator

It is used to combine the result of two different tables removing duplicate rows.

3. Union All Operator

It is used to combine the result of two or more 'select' statement. It includes duplicate values too.

4. Group by Statement

It is used to combine rows having the same value into group; usually so we can perform calculation such as COUNT(), SUM(), AVG(), MAX(), MIN() on each group.

5. Having

It is used to filter groups created by Group By



Mo	Tu	We	Th	Fr	Sa	Su
----	----	----	----	----	----	----

Memo No. _____

Date / /

for eg:

```
SELECT Department, AVG(Marks) AS  
AverageMarks  
FROM Students  
GROUP BY Department  
HAVING AVG(Marks) > 80;
```

6. Exists

It checks whether a subquery returns a least one row. It doesn't really care what the returned value is it only ask does at least one matching record exist.

7. Any

It compares a value with the values returned by a subquery and is true if the condition is true for at least one value

8. ALL

It compares a value with every value returned by a subquery.

⊗ Any = At least one

All = Every one.

9. SELECT INTO

It is used to create a new table and copy data into it from an existing table.

Select *

INTO student Backup

From Student;

10. Insert Into Select

It copies the data from one existing table to ~~another~~ another existing table. For eg. ~~the~~ student table, graduated student table. Moving student table into existing graduated student table once they are being graduated.

④ Select Into = Create + copy

⑤ Insert Into Select = Existing table + copy

11. Case

It is used to perform conditional logic inside SQL like if, else if.

For eg: You have a table named students with their names and marks. Now You need to give 90 - A+, 80 - A, 70 - B+, 60 - B--

Select Name, Marks,
Case

When Marks ≥ 90 Then 'A'

When Marks ≥ 80 Then 'A'

When Marks ≥ 70 Then 'AB'

When Marks ≥ 60 Then 'B'

END AS Result

From Students;

12. Null Functions

Null means missing, unknown, or unavailable data. SQL provides functions to handle null values. like COALESCE() - which will replace null value with another value

Coalesce (value, replacement)

13. Comments

These are the text written inside SQL code to explain the code. That are not be executed

Single line comment: - -

Multi line comment: /* */

14. Subquery

The ~~so~~ SQL query written inside another SQL query, where inner query provides information that the outer query uses.

Select *

From student

Where Mark > (

Select AVG (Marks)

From student

);

15. CTE (Common Table Expression)

It is a temporary named result that you create at the beginning of a SQL query and then use in the main query.

it ~~is~~ can be created using WITH
freq.

16. With Expensive Products As

(

Select *

FROM Products

Where Price > 1000

)

Select *

From ExpensiveProducts

Where Category = 'Lap top';

16. Stored Procedure

It is a saved group of SQL commands that you can run whenever you need. for eg

Create PROCEDURE GetEmployees

As

Begin

Select *

From Employees. [= 2]

END.

EXEC GetEmployees

It also can ~~a~~ parameters like

[(a) EmployeeID Int @
= @EmployeeID]

Then execute ~~id~~ by giving multiple parameters

EXEC GetEmployee @EmployeeID = 10;

```
1  • SELECT salary FROM Teacher ORDER BY salary DESC LIMIT 1,1;
2
3
4  • SELECT AVG(salary) As AverageTeacherSalary FROM Teacher;
5
6  • SELECT MIN(salary) As MinumTeacherSalary, MAX(salary) AS MaximumTeacherSalary FROM Teacher;
7
8  • SELECT * FROM Teacher;
9
10 • SELECT * FROM Departement;
11
12 • SELECT s.StudentId, s.StudentFirstName, s.StudentLastName, d.DepertementName
13    FROM Students s INNER JOIN Departement d ON s.DepartmentID = d.DepertementId;
14
15 • SELECT d.DepertementId, d.DepertementName,s.StudentFirstName
16    FROM Departement d LEFT JOIN Students s ON d.DepertementId = s.DepartmentID;
17
18 • SELECT d.DepertementName,COUNT(s.StudentId) AS TotalStudents
19    FROM Departement d LEFT JOIN Students s ON d.DepertementId = s.DepartmentID
20    GROUP BY d.DepertementId, d.DepertementName
21    ORDER BY TotalStudents DESC;
22
23 • SELECT StudentFirstName, StudentLastName,
24    CASE
25    WHEN IsFeePaid = 1 THEN 'Paid'
```

```
WHEN IsFeePaid = 1 THEN 'Paid'
```

Output	Result #1	Result #2	Result #3	Result #4	Result #5
StudentFirstName	StudentLastName		FeeStatus		
Aarav	Sharma		Paid		
Sita	Thapa		Paid		
Bibek	Gurung		Not Paid		
Anisha	Karki		Paid		
Rohan	Adhikari		Not Paid		
Pratik	Poudel		Paid		
Samiksha	Shrestha		Paid		
Nabin	Bista		Not Paid		
Sneha	Rai		Paid		
Roshan	Magar		Not Paid		

OK, 30 records retrieved in 0.732ms